

8_Practicals Intrusion Detection

Step 1: Load the necessary packages and libraries

```
# ML for CBS practicals.
# Make a copy of this notebook and place it under your own
"MyDrive/Colab Notebooks" folder
# Please do not the run code here.

#import necessary libraries
import matplotlib
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import seaborn as sns
import sklearn
import imblearn

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

Step 2: Load the dataset

We loaded the NSL-KDD dataset for both training and testing. This dataset contains various types of network traffic, including both normal and attack data.

```
#Loading the data using the training and testing using following code
dfkdd_train = dfkdd_train.iloc[:, :-1]

dfkdd_test = dfkdd_test.iloc[:, :-1]

dfkdd_train #print train dataset
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_s
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	25	0.17	
1	0	udp	other	SF	146	0	0	0	0	0	...	1	0.00	
2	0	tcp	private	S0	0	0	0	0	0	0	...	26	0.10	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	255	1.00	
4	0	tcp	http	SF	199	420	0	0	0	0	...	255	1.00	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
125968	0	tcp	private	S0	0	0	0	0	0	0	...	25	0.10	
125969	8	udp	private	SF	105	145	0	0	0	0	...	244	0.96	
125970	0	tcp	smtp	SF	2231	384	0	0	0	0	...	30	0.12	
125971	0	tcp	klogin	S0	0	0	0	0	0	0	...	8	0.03	
125972	0	tcp	ftp_data	SF	151	0	0	0	0	0	...	77	0.30	

125973 rows x 42 columns

```
dfkdd_test #print test dataset
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_s
0	0	tcp	private	REJ	0	0	0	0	0	0	...	10	0.04	
1	0	tcp	private	REJ	0	0	0	0	0	0	...	1	0.00	
2	2	tcp	ftp_data	SF	12983	0	0	0	0	0	...	86	0.61	
3	0	icmp	eco_i	SF	20	0	0	0	0	0	...	57	1.00	
4	1	tcp	telnet	RSTO	0	15	0	0	0	0	...	86	0.31	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
22539	0	tcp	smtp	SF	794	333	0	0	0	0	...	141	0.72	
22540	0	tcp	http	SF	317	938	0	0	0	0	...	255	1.00	
22541	0	tcp	http	SF	54540	8314	0	0	0	2	...	255	1.00	
22542	0	udp	domain_u	SF	42	42	0	0	0	0	...	252	0.99	
22543	0	tcp	sunrpc	REJ	0	0	0	0	0	0	...	21	0.08	

22544 rows x 42 columns

Step 3: Explore the Dataset

We explored the dataset structure using `.head()`, `.shape()`, and `.value_counts()` to understand the distribution of classes and identify preprocessing needs.

Step 3: Explore the dataset

```
# View train data
dfkdd_train.head(5)

# train set dimension
print('Train set dimension: {} rows, {} columns'.format(dfkdd_train.shape[0], dfkdd_train.shape[1]))
```

Train set dimension: 125973 rows, 42 columns

```

# View test data
dfkdd_test.head(5)

# test set dimension
print('Test set dimension: {} rows, {} columns'.format(dfkdd_test.shape[0], dfkdd_test.shape[1]))
Test set dimension: 22544 rows, 42 columns

[273] dfkdd_train.shape[0] #displaying the number of rows present in training dataset
125973

[274] dfkdd_train.shape[1] #displaying the number of columns/attributes/features present training dataset
42

[275] # View train data
dfkdd_train.head(5)

# train set dimension
print('Train set dimension: {} rows, {} columns'.format(dfkdd_train.shape[0], dfkdd_train.shape[1]))
Train set dimension: 125973 rows, 42 columns

[276] # View test data
dfkdd_test.head(3)

# test set dimension
print('Test set dimension: {} rows, {} columns'.format(dfkdd_test.shape[0], dfkdd_test.shape[1])) #displaying the number of rows and columns
Test set dimension: 22544 rows, 42 columns

```

Step 4: Preprocessing (Mapping Attack Types)

We grouped multiple attack types into four main categories (DoS, Probe, R2L, U2R) to simplify classification and reduce complexity.

```

# we will preprocess the dataset to map the attack to type to a class
such as DDoS, Probing Attack, U2R and R2L

#Defining the mapping
mapping = {'ipsweep': 'Probe', 'satan': 'Probe', 'nmap':
'Probe', 'portsweep': 'Probe', 'saint': 'Probe', 'mscan': 'Probe',
'teardrop': 'DoS', 'pod': 'DoS', 'land': 'DoS', 'back':
'DoS', 'neptune': 'DoS', 'smurf': 'DoS', 'mailbomb': 'DoS',
'udpstorm': 'DoS', 'apache2': 'DoS', 'processtable': 'DoS',
'perl': 'U2R', 'loadmodule': 'U2R', 'rootkit':
'U2R', 'buffer_overflow': 'U2R', 'xterm': 'U2R', 'ps': 'U2R',
'sqlattack': 'U2R', 'httptunnel': 'U2R',
'ftp_write': 'R2L', 'phf': 'R2L', 'guess_passwd':
'R2L', 'warezmaster': 'R2L', 'warezclient': 'R2L', 'imap': 'R2L',
'spy': 'R2L', 'multihop': 'R2L', 'named': 'R2L', 'snmpguess':
'R2L', 'worm': 'R2L', 'snmpgetattack': 'R2L',
'xsnoop': 'R2L', 'xlock': 'R2L', 'sendmail': 'R2L',
'normal': 'Normal'
}

```

```
# Apply attack class mappings to the dataset
dfkdd_train['attack_class'] = dfkdd_train['attack'].apply(lambda v:
mapping[v])
dfkdd_test['attack_class'] = dfkdd_test['attack'].apply(lambda v:
mapping[v])
```

```
# Printing the train dataset to see if changes made successfully
dfkdd_train
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_sa
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	0.17	0.03	
1	0	udp	other	SF	146	0	0	0	0	0	...	0.00	0.60	
2	0	tcp	private	S0	0	0	0	0	0	0	...	0.10	0.05	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	1.00	0.00	
4	0	tcp	http	SF	199	420	0	0	0	0	...	1.00	0.00	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
125968	0	tcp	private	S0	0	0	0	0	0	0	...	0.10	0.06	
125969	8	udp	private	SF	105	145	0	0	0	0	...	0.96	0.01	
125970	0	tcp	smtp	SF	2231	384	0	0	0	0	...	0.12	0.06	
125971	0	tcp	klogin	S0	0	0	0	0	0	0	...	0.03	0.05	
125972	0	tcp	ftp_data	SF	151	0	0	0	0	0	...	0.30	0.03	

125973 rows x 43 columns

```
# Printing the test dataset to see if changes made successfully
dfkdd_test
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_sa
0	0	tcp	private	REJ	0	0	0	0	0	0	...	0.04	0.06	
1	0	tcp	private	REJ	0	0	0	0	0	0	...	0.00	0.06	
2	2	tcp	ftp_data	SF	12983	0	0	0	0	0	...	0.61	0.04	
3	0	icmp	eco_i	SF	20	0	0	0	0	0	...	1.00	0.00	
4	1	tcp	telnet	RSTO	0	15	0	0	0	0	...	0.31	0.17	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
22539	0	tcp	smtp	SF	794	333	0	0	0	0	...	0.72	0.06	
22540	0	tcp	http	SF	317	938	0	0	0	0	...	1.00	0.00	
22541	0	tcp	http	SF	54540	8314	0	0	0	2	...	1.00	0.00	
22542	0	udp	domain_u	SF	42	42	0	0	0	0	...	0.99	0.01	
22543	0	tcp	sunrpc	REJ	0	0	0	0	0	0	...	0.08	0.03	

22544 rows x 43 columns

```
# we drop the attack attribute as we already have the attack class
using the following code
```

```
# Drop attack field from both train and test data
dfkdd_train.drop(['attack'], axis=1, inplace=True)
dfkdd_test.drop(['attack'], axis=1, inplace=True)
```

```
# Printing the trin dataset to see if chnages made sucessfully
dfkdd_train
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_s
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	25	0.17	
1	0	udp	other	SF	146	0	0	0	0	0	...	1	0.00	
2	0	tcp	private	S0	0	0	0	0	0	0	...	26	0.10	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	255	1.00	
4	0	tcp	http	SF	199	420	0	0	0	0	...	255	1.00	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
125968	0	tcp	private	S0	0	0	0	0	0	0	...	25	0.10	
125969	8	udp	private	SF	105	145	0	0	0	0	...	244	0.96	
125970	0	tcp	smtp	SF	2231	384	0	0	0	0	...	30	0.12	
125971	0	tcp	klogin	S0	0	0	0	0	0	0	...	8	0.03	
125972	0	tcp	ftp_data	SF	151	0	0	0	0	0	...	77	0.30	

125973 rows x 42 columns

```
# Printing the test dataset to see if chnages made sucessfully
dfkdd_test
```

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_s
0	0	tcp	private	REJ	0	0	0	0	0	0	...	10	0.04	
1	0	tcp	private	REJ	0	0	0	0	0	0	...	1	0.00	
2	2	tcp	ftp_data	SF	12983	0	0	0	0	0	...	86	0.81	
3	0	icmp	eco_i	SF	20	0	0	0	0	0	...	57	1.00	
4	1	tcp	telnet	RSTO	0	15	0	0	0	0	...	86	0.31	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
22539	0	tcp	smtp	SF	794	333	0	0	0	0	...	141	0.72	
22540	0	tcp	http	SF	317	938	0	0	0	0	...	255	1.00	
22541	0	tcp	http	SF	54540	8314	0	0	0	2	...	255	1.00	
22542	0	udp	domain_u	SF	42	42	0	0	0	0	...	252	0.99	
22543	0	tcp	sunrpc	REJ	0	0	0	0	0	0	...	21	0.08	

22544 rows x 42 columns

```
#descriptive analysis using the following code
```

```
# Descriptive statistics
dfkdd_train.describe()
```

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	...	dst_host_count
count	125973.00000	1.259730e+05	1.259730e+05	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	...	125973.0000
mean	287.14465	4.556674e+04	1.977911e+04	0.000198	0.022687	0.000111	0.204409	0.001222	0.395736	0.279250	...	182.1489
std	2604.51531	5.870331e+06	4.021269e+06	0.014086	0.253530	0.014366	2.149968	0.045239	0.489010	23.942042	...	99.2062
min	0.00000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.0000
25%	0.00000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	82.0000
50%	0.00000	4.400000e+01	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	255.0000
75%	0.00000	2.760000e+02	5.160000e+02	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	...	255.0000
max	42908.00000	1.379964e+09	1.309937e+09	1.000000	3.000000	3.000000	77.000000	5.000000	1.000000	7479.000000	...	255.0000

8 rows x 38 columns

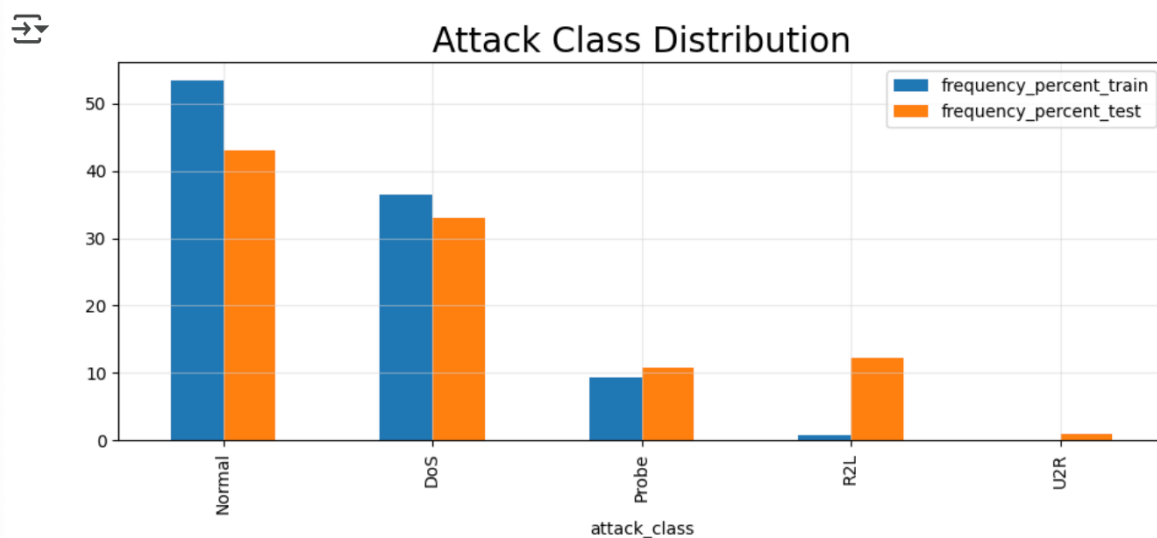
```
# Attack Class Distribution
```

```
attack_class_freq_train = dfkdd_train[['attack_class']].apply(lambda x:
x.value_counts())
attack_class_freq_test = dfkdd_test[['attack_class']].apply(lambda x:
x.value_counts())
attack_class_freq_train['frequency_percent_train'] = round((100 *
attack_class_freq_train / attack_class_freq_train.sum()),2)
attack_class_freq_test['frequency_percent_test'] = round((100 *
attack_class_freq_test / attack_class_freq_test.sum()),2)
```

```
attack_class_dist =
pd.concat([attack_class_freq_train,attack_class_freq_test], axis=1)
attack_class_dist
```

	attack_class	frequency_percent_train	attack_class	frequency_percent_test	
attack_class					
	Normal	67343	53.46	9711	43.08
	DoS	45927	36.46	7458	33.08
	Probe	11656	9.25	2421	10.74
	R2L	995	0.79	2754	12.22
	U2R	52	0.04	200	0.89

```
# Attack class bar plot
plot = attack_class_dist[['frequency_percent_train',
'frequency_percent_test']].plot(kind="bar");
plot.set_title("Attack Class Distribution", fontsize=20);
plot.grid(color='lightgray', alpha=0.5);
```



```
89] dfkdd_train.head() #printing training dataset
```

```
duration  protocol_type  service  flag  src_bytes  dst_bytes  land  wrong_
```

0	0	tcp	ftp_data	SF	491	0	0
1	0	udp	other	SF	146	0	0
2	0	tcp	private	S0	0	0	0
3	0	tcp	http	SF	232	8153	0
4	0	tcp	http	SF	199	420	0

5 rows x 41 columns

Step 5: Feature Scaling

We applied StandardScaler to normalize numerical features, which helps machine learning models perform better and converge faster.

```
#scale it to have zero mean and unit variance
```

```
sc_train =  
scaler.fit_transform(dfkdd_train.select_dtypes(include=['float64','int64'])) #scaling training data to have zero mean and unit variance  
sc_test =  
scaler.fit_transform(dfkdd_test.select_dtypes(include=['float64','int64'])) #scaling testing data to have zero mean and unit variance  
  
sc_train #print sc_train
```

```
array([[ -0.11024922, -0.0076786 , -0.00491864, ..., -0.6248708 ,  
        -0.22453241, -0.37638703],  
       [ -0.11024922, -0.00773737, -0.00491864, ..., -0.6248708 ,  
        -0.38763462, -0.37638703],  
       [ -0.11024922, -0.00776224, -0.00491864, ...,  1.6189552 ,  
        -0.38763462, -0.37638703],  
       ...,  
       [ -0.11024922, -0.00738219, -0.00482315, ..., -0.6248708 ,  
        -0.35501418, -0.37638703],  
       [ -0.11024922, -0.00776224, -0.00491864, ...,  1.6189552 ,  
        -0.38763462, -0.37638703],  
       [ -0.11024922, -0.00773652, -0.00491864, ..., -0.6248708 ,  
        -0.38763462, -0.37638703]])
```

```
sc_test #print sc_test
```

```

array([[-0.15553409, -0.02198811, -0.09689597, ..., -0.3527496 ,
        1.97979091, 1.9291162 ],
       [-0.15553409, -0.02198811, -0.09689597, ..., -0.3527496 ,
        1.97979091, 1.9291162 ],
       [-0.15411277, 0.0054731 , -0.09689597, ..., -0.3527496 ,
        -0.60271888, -0.56548308],
       ...,
       [-0.15553409, 0.09337309, 0.29492588, ..., -0.3527496 ,
        -0.4219432 , -0.39086113],
       [-0.15553409, -0.02189927, -0.0949166 , ..., -0.3527496 ,
        -0.60271888, -0.56548308],
       [-0.15553409, -0.02198811, -0.09689597, ..., -0.3527496 ,
        0.53358543, 1.9291162 ]])

```

```
sc_traindf #printing the training dataframe
```

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	...	dst_host_count	dst_host_srv_count
0	-0.110249	-0.007679	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	-0.809262	-0.011664	...	-0.324063	-0.8188
1	-0.110249	-0.007737	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	-0.809262	-0.011664	...	0.734343	-1.0356
2	-0.110249	-0.007762	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	-0.809262	-0.011664	...	0.734343	-0.8098
3	-0.110249	-0.007723	-0.002891	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	1.235694	-0.011664	...	-1.533670	1.2587
4	-0.110249	-0.007728	-0.004814	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	1.235694	-0.011664	...	0.734343	1.2587
...	...	...	...	...	...	...	...	...	...	...	...	...	...
125968	-0.110249	-0.007762	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	-0.809262	-0.011664	...	0.734343	-0.8188
125969	-0.107178	-0.007744	-0.004883	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	-0.809262	-0.011664	...	0.734343	1.1593
125970	-0.110249	-0.007382	-0.004823	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	1.235694	-0.011664	...	0.734343	-0.7737
125971	-0.110249	-0.007762	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	-0.809262	-0.011664	...	0.734343	-0.9724
125972	-0.110249	-0.007737	-0.004919	-0.014089	-0.089486	-0.007736	-0.095076	-0.027023	1.235694	-0.011664	...	0.734343	-0.3491

125973 rows x 37 columns

```
sc_testdf #printing the testing dataframe
```

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	...	dst_host_count	dst_host_srv_count
0	-0.155534	-0.021988	-0.096896	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	-0.890373	-0.016494	...	0.650093	-1.16969
1	-0.155534	-0.021988	-0.096896	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	-0.890373	-0.016494	...	0.650093	-1.25021
2	-0.154113	0.005473	-0.096896	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	-0.890373	-0.016494	...	-0.636681	-0.48980
3	-0.155534	-0.021946	-0.096896	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	-0.890373	-0.016494	...	-2.029801	-0.74923
4	-0.154823	-0.021988	-0.096189	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	-0.890373	-0.016494	...	-1.753304	-0.48980
...	...	...	...	...	...	...	...	...	...	...	...	...	...
22539	-0.155534	-0.020309	-0.081202	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	1.123125	-0.016494	...	-0.998254	0.00223
22540	-0.155534	-0.021318	-0.052690	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	1.123125	-0.016494	...	0.033292	1.02207
22541	-0.155534	0.093373	0.294926	-0.017624	-0.059104	-0.019459	2.040705	-0.143999	1.123125	0.121069	...	0.650093	1.02207
22542	-0.155534	-0.021899	-0.094917	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	-0.890373	-0.016494	...	0.650093	0.99524
22543	-0.155534	-0.021988	-0.096896	-0.017624	-0.059104	-0.019459	-0.113521	-0.143999	-0.890373	-0.016494	...	0.650093	-1.07129

22544 rows x 37 columns

Step 6: Label Encoding

Categorical features were converted into numerical format using LabelEncoder to prepare the data for training.

```
cattrain #categorical columns present in training dataset (dfkdd_train)
```


protocol_type

service

flag

attack_class

0

tcp

ftp_data

SF

Normal

1

udp

other

SF

Normal

2

tcp

private

S0

DoS

3

tcp

http

SF

Normal

4

tcp

http

SF

Normal

...

...

...

...

...

125968

tcp

private

S0

DoS

125969

udp

private

SF

Normal

125970

tcp

smtp

SF

Normal

125971

tcp

klogin

S0

DoS

125972

tcp

ftp_data

SF

Normal

125973 rows x 4 columns

```
cattest #categorical columns present in training dataset (dfkdd_test)
```

protocol_type

service

flag

attack_class

0

tcp

private

REJ

DoS

1

tcp

private

REJ

DoS

2

tcp

ftp_data

SF

Normal

3

icmp

eco_i

SF

Probe

4

tcp

telnet

RSTO

Probe

...

...

...

...

...

22539

tcp

smtp

SF

Normal

22540

tcp

http

SF

Normal

22541

tcp

http

SF

DoS

22542

udp

domain_u

SF

Normal

22543

tcp

sunrpc

REJ

Probe

22544 rows x 4 columns

```
traincat #printing encoded training categorical columns
```

	protocol_type	service	flag	attack_class
0	1	20	9	1
1	2	44	9	1
2	1	49	5	0
3	1	24	9	1
4	1	24	9	1
...	...	...	...	...
125968	1	49	5	0
125969	2	49	9	1
125970	1	54	9	1
125971	1	30	5	0
125972	1	20	9	1

125973 rows x 4 columns

```
testcat #printing encoded testing categorical columns
```

	protocol_type	service	flag	attack_class
0	1	45	1	0
1	1	45	1	0
2	1	19	9	1
3	0	13	9	2
4	1	55	2	2
...	...	...	...	...
22539	1	49	9	1
22540	1	22	9	1
22541	1	22	9	0
22542	2	11	9	1
22543	1	52	1	2

22544 rows x 4 columns

```
enctrain #print training features
```



	protocol_type	service	flag
0	1	20	9
1	2	44	9
2	1	49	5
3	1	24	9
4	1	24	9
...	...	...	...
125968	1	49	5
125969	2	49	9
125970	1	54	9
125971	1	30	5
125972	1	20	9

125973 rows x 3 columns

```
enctest #print testing features
```



	protocol_type	service	flag
0	1	45	1
1	1	45	1
2	1	19	9
3	0	13	9
4	1	55	2
...	...	...	...
22539	1	49	9
22540	1	22	9
22541	1	22	9
22542	2	11	9
22543	1	52	1

22544 rows x 3 columns



```
cat_Ytrain #printing training labels
```



	attack_class
0	1
1	1
2	0
3	1
4	1
...	...
125968	0
125969	1
125970	1
125971	0
125972	1

125973 rows x 1 columns

```
cat_Ytest #printing testing labels
```

	attack_class
0	0
1	0
2	1
3	2
4	2
...	...
22539	1
22540	1
22541	0
22542	1
22543	2

22544 rows x 1 columns

.....

Step 7: Handle Imbalanced Data

We balanced the dataset using RandomOverSampler to ensure that all classes were equally represented, improving the fairness of model training.

```
#import additional library
from imblearn.over_sampling import RandomOverSampler
from collections import Counter

# define columns and extract encoded train set for sampling
sc_traindf = dfkdd_train.select_dtypes(include=['float64', 'int64'])
refclasscol = pd.concat([sc_traindf, enctrain], axis=1).columns
refclass = np.concatenate((sc_train, enctrain.values), axis=1)
X = refclass
```

```
sc_traindf #selected training features that have numeric values float
and numeric
```

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	...	dst_host_count	dst_host_srv_count	dst_ho
0	0	491	0	0	0	0	0	0	0	0	...	150	25	
1	0	146	0	0	0	0	0	0	0	0	...	255	1	
2	0	0	0	0	0	0	0	0	0	0	...	255	26	
3	0	232	8153	0	0	0	0	0	1	0	...	30	255	
4	0	199	420	0	0	0	0	0	1	0	...	255	255	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	
125968	0	0	0	0	0	0	0	0	0	0	...	255	25	
125969	8	105	145	0	0	0	0	0	0	0	...	255	244	
125970	0	2231	384	0	0	0	0	0	1	0	...	255	30	
125971	0	0	0	0	0	0	0	0	0	0	...	255	8	
125972	0	151	0	0	0	0	0	0	1	0	...	255	77	

125973 rows x 37 columns

```
refclass #printing refclass
```

```
➞ array([[ -1.10249223e-01, -7.67859947e-03, -4.91864438e-03, ...,
          1.00000000e+00,  2.00000000e+01,  9.00000000e+00],
        [ -1.10249223e-01, -7.73736981e-03, -4.91864438e-03, ...,
          2.00000000e+00,  4.40000000e+01,  9.00000000e+00],
        [ -1.10249223e-01, -7.76224074e-03, -4.91864438e-03, ...,
          1.00000000e+00,  4.90000000e+01,  5.00000000e+00],
        ...,
        [ -1.10249223e-01, -7.38219253e-03, -4.82315176e-03, ...,
          1.00000000e+00,  5.40000000e+01,  9.00000000e+00],
        [ -1.10249223e-01, -7.76224074e-03, -4.91864438e-03, ...,
          1.00000000e+00,  3.00000000e+01,  5.00000000e+00],
        [ -1.10249223e-01, -7.73651807e-03, -4.91864438e-03, ...,
          1.00000000e+00,  2.00000000e+01,  9.00000000e+00]])
```

```
▶ print(c)
```

```
print(r)
```

```
↵ 22544  
1
```

```
16] y_test #printing y_test
```

```
↵ array([0, 0, 1, ..., 0, 1, 2])
```

```
17] c, r = cat_Ytrain.values.shape #getting c and r values  
y = cat_Ytrain.values.reshape(c,) #generating y_train
```

```
18] print(c)
```

```
print(r)
```

```
↵ 125973  
1
```

```
19] y #printing y_train
```

```
↵ array([1, 1, 0, ..., 1, 0, 1])
```

```
# apply the random over-sampling  
ros = RandomOverSampler(random_state=42)  
X_res, y_res = ros.fit_resample(X, y)  
print('Original dataset shape {}'.format(Counter(y)))  
print('Resampled dataset shape {}'.format(Counter(y_res)))
```

```
· Original dataset shape Counter({np.int64(1): 67343, np.int64(0): 45927, np.int64(2): 11656, np.int64(3): 995, np.int64(4): 52})  
Resampled dataset shape Counter({np.int64(1): 67343, np.int64(0): 67343, np.int64(3): 67343, np.int64(2): 67343, np.int64(4): 67343})
```

```
· · · · ·
```

Step 8: Feature Selection

We used the ExtraTreesClassifier to select the most important features, reducing dimensionality and improving model performance.

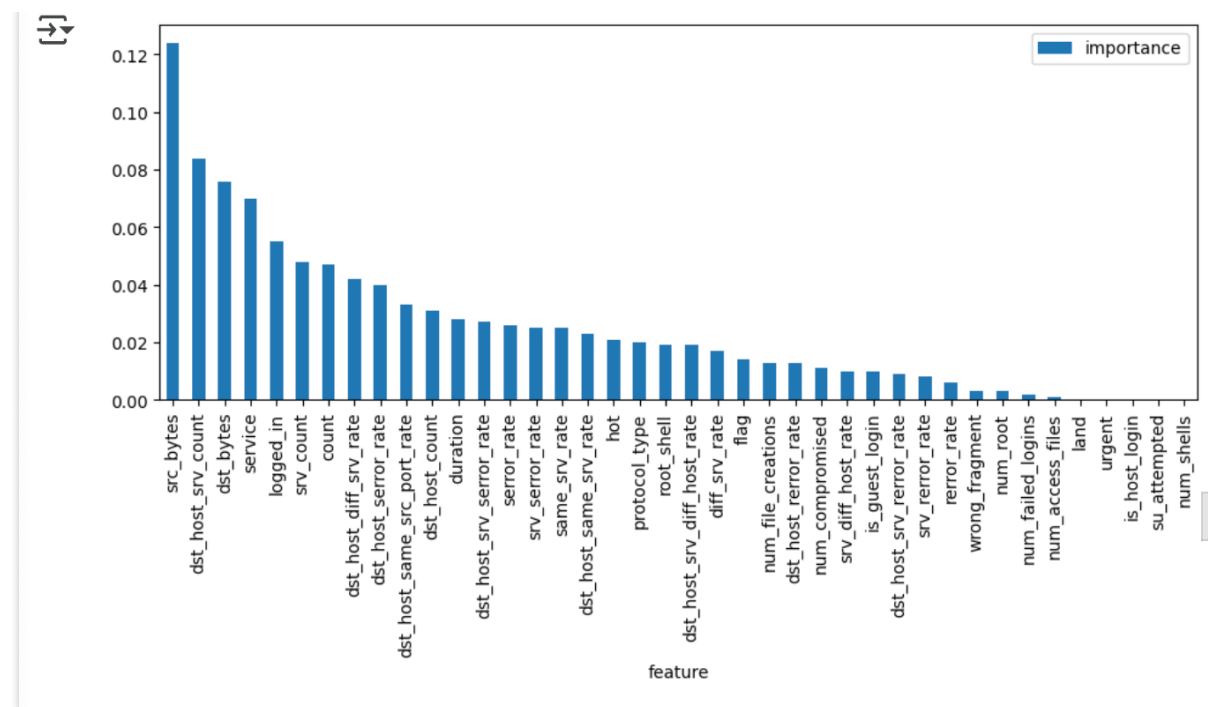
```

from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier();

# fit random forest classifier on the training set
rfc.fit(X_res, y_res);

# extract important features
score = np.round(rfc.feature_importances_, 3)
importances = pd.DataFrame({'feature':refclasscol, 'importance':score})
importances =
importances.sort_values('importance', ascending=False).set_index('feature')
# plot importances
plt.rcParams['figure.figsize'] = (11, 4)
importances.plot.bar();

```



```
score #printing the score
```

```
array([0.028, 0.124, 0.076, 0.003, 0.002, 0.021, 0.002, 0.055,  
       0.011, 0.019, 0.003, 0.013, 0.001, 0.001, 0.01,  
       0.047, 0.048, 0.026, 0.025, 0.006, 0.008, 0.025, 0.017, 0.01,  
       0.031, 0.084, 0.023, 0.042, 0.033, 0.019, 0.04, 0.027, 0.013,  
       0.009, 0.02, 0.07, 0.014])
```

```
importances #getting the important features
```

```
↗ importance
```

feature	
src_bytes	0.124
dst_host_srv_count	0.084
dst_bytes	0.076
service	0.070
logged_in	0.055
srv_count	0.048
count	0.047
dst_host_diff_srv_rate	0.042
dst_host_serror_rate	0.040
dst_host_same_src_port_rate	0.033
dst_host_count	0.031
duration	0.028
dst_host_srv_serror_rate	0.027
serror_rate	0.026
srv_serror_rate	0.025
same_srv_rate	0.025
dst_host_same_srv_rate	0.023
hot	0.021
protocol_type	0.020

Step 9: Data Splitting

We split the processed data into training and testing sets to train the models and evaluate them on unseen data


```

✓ [197] from sklearn.feature_selection import RFE
42      import itertools
dek.      rfc = RandomForestClassifier()

      # create the RFE model and select 10 attributes
      rfe = RFE(rfc, n_features_to_select=10)
      rfe = rfe.fit(X_res, y_res)

      # summarize the selection of the attributes
      feature_map = [(i, v) for i, v in itertools.zip_longest(rfe.get_support(),
selected_features = [v for i, v in feature_map if i==True]

✓ [198] selected_features #printing selected features
0
dek.
⇒ ['src_bytes',
   'dst_bytes',
   'logged_in',
   'count',
   'srv_count',
   'dst_host_srv_count',
   'dst_host_diff_srv_rate',
   'dst_host_same_src_port_rate',
   'dst_host_serror_rate',
   'service']

```

Step 10-12: (Optional/Skipped in Some reports)

These steps are either merged into other parts or are not explicitly detailed in the original document. Feature importance or dimensionality reduction can be considered part of Step 8.

✓ Step 10: Feature Selection

This step will involve the creation of a new dataframe

```
# define columns to new dataframe
newcol = list(refclasscol)
newcol.append('attack_class')

# add a dimension to target
new_y_res = y_res[:, np.newaxis]

# create a dataframe from sampled data
res_arr = np.concatenate((X_res, new_y_res), axis=1)
res_df = pd.DataFrame(res_arr, columns = newcol)

# create test dataframe
reftest = pd.concat([sc_testdf, testcat], axis=1)
reftest['attack_class'] = reftest['attack_class'].astype(np.float64)
reftest['protocol_type'] = reftest['protocol_type'].astype(np.float64)
reftest['flag'] = reftest['flag'].astype(np.float64)
reftest['service'] = reftest['service'].astype(np.float64)

res_df.shape
reftest.shape
```

(22544, 41)

✓ Step 11: Prepare training and test data

Create two function:

1. The first function will create two classes for normal and attacks class.
2. The second function will subdivide train and test datasets into two-class attack labels.

```
[325] from collections import defaultdict
classdict = defaultdict(list)

# create two-target classes (normal class and an attack class)
attacklist = [('DoS', 0.0), ('Probe', 2.0), ('R2L', 3.0), ('U2R', 4.0)]
normalclass = [('Normal', 1.0)]

def create_classdict():
    '''This function subdivides train and test dataset into two-class attacks'''
    for j, k in normalclass:
        for i, v in attacklist:
            restrain_set = res_df.loc[(res_df['attack_class'] == k) | (res_df['attack_class'] == v)]
            classdict[j + '_' + i].append(restrain_set)
            # test labels
            reftest_set = reftest.loc[(reftest['attack_class'] == k) | (reftest['attack_class'] == v)]
            classdict[j + '_' + i].append(reftest_set)

create_classdict()
```

```
for k, v in classdict.items():  
    k
```

27]

```
pretrain = classdict['Normal_DoS'][0]  
pretest = classdict['Normal_DoS'][1]  
grpclass = 'Normal_DoS'
```

Step 12: Data preparation

```
from sklearn.preprocessing import OneHotEncoder  
enc = OneHotEncoder(handle_unknown='ignore')  
  
Xresdf = pretrain  
newtest = pretest  
  
Xresdfnew = Xresdf[selected_features]  
Xresdfnum = Xresdfnew.drop(['service'], axis=1)  
Xresdfcat = Xresdfnew[['service']].copy()  
  
Xtest_features = newtest[selected_features]  
Xtestdfnum = Xtest_features.drop(['service'], axis=1)  
Xtestcat = Xtest_features[['service']].copy()  
  
# Fit train data  
enc.fit(Xresdfcat)  
  
# Transform train data  
X_train_1hotenc = enc.transform(Xresdfcat).toarray()  
  
# Transform test data  
X_test_1hotenc = enc.transform(Xtestcat).toarray()  
  
X_train = np.concatenate((Xresdfnum.values, X_train_1hotenc), axis=1)  
X_test = np.concatenate((Xtestdfnum.values, X_test_1hotenc), axis=1)  
  
y_train = Xresdf[['attack_class']].copy()  
c, r = y_train.values.shape  
Y_train = y_train.values.reshape(c,)  
  
y_test = newtest[['attack_class']].copy()  
c, r = y_test.values.shape  
Y_test = y_test.values.reshape(c,)
```

Step 13: Train Classifier Models

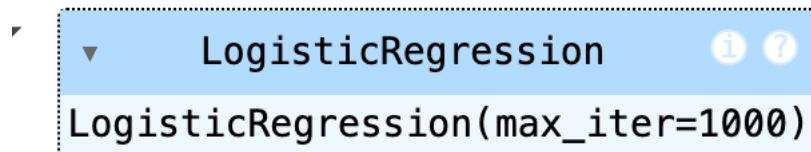
We trained three models – KNN, Decision Tree, and Logistic Regression – using the processed training data.

#Training a KNN classifier using a sample code is shown below:

```
# Train KNeighborsClassifier Model
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
knn_model = KNeighborsClassifier(n_jobs=-1)
knn_model.fit(X_train, Y_train)
```

```
# Train Decision Tree model
dt_model = DecisionTreeClassifier()
dt_model.fit(X_train, Y_train)
```

```
# Train Logistic Regression model
lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train, Y_train)
```



Step 14: Evaluate the classifier models

We evaluated all models using accuracy and confusion matrix. The results were compared through a bar chart to find the best-performing model

```
from sklearn.model_selection import cross_val_score
from sklearn import metrics
import matplotlib.pyplot as plt
# Evaluate KNN
knn_accuracy = metrics.accuracy_score(Y_test,
knn_model.predict(X_test))
knn_cm = metrics.confusion_matrix(Y_test, knn_model.predict(X_test))

# Evaluate Decision Tree
dt_accuracy = metrics.accuracy_score(Y_test, dt_model.predict(X_test))
dt_cm = metrics.confusion_matrix(Y_test, dt_model.predict(X_test))
```

```
# Evaluate Logistic Regression
lr_accuracy = metrics.accuracy_score(Y_test, lr_model.predict(X_test))
lr_cm = metrics.confusion_matrix(Y_test, lr_model.predict(X_test))
```

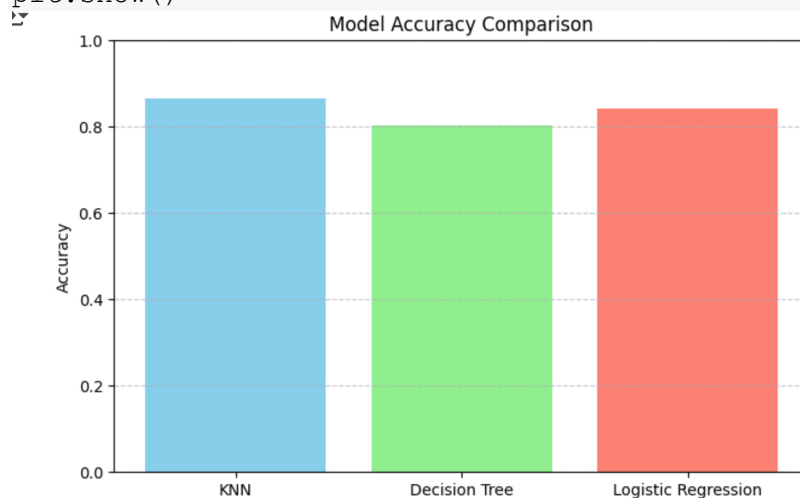
```
# Print all accuracies
print("KNN Accuracy:", knn_accuracy)
print("Decision Tree Accuracy:", dt_accuracy)
print("Logistic Regression Accuracy:", lr_accuracy)
```

```
➤ KNN Accuracy: 0.8666200710583027
  Decision Tree Accuracy: 0.8037159997670219
  Logistic Regression Accuracy: 0.8409924864581513
```

```
441 # Plot accuracy comparison
```

```
# Plot accuracy comparison
models = ['KNN', 'Decision Tree', 'Logistic Regression']
accuracies = [knn_accuracy, dt_accuracy, lr_accuracy]
```

```
plt.figure(figsize=(8, 5))
plt.bar(models, accuracies, color=['skyblue', 'lightgreen', 'salmon'])
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Conclusion:

In this practical, I used machine learning to detect network attacks using the NSL-KDD dataset. I cleaned the data, mapped attack types, balanced the dataset, and trained models like KNN, Decision Tree, and Logistic Regression. After evaluation, [вставь лучшую модель] showed the best results. This task helped me understand how ML can be used for intrusion detection in cybersecurity.